Brief Quest: Architecture and Learning Notes

A companion pack for the 4-week Brief Quest plan: the concept pictures from the ASP.NET Core tutorial, the planned architecture, tables about the code, and my evidence log. Prepared 19 October 2026.

How to read this pack

- **Figures and tables are numbered.** Each one has a description and a "Belongs to" line naming the tutorial session (S1 to S4) or the plan days it supports.
- **Colours mean the same thing throughout.** In the architecture figures: pink is the front end, amber is the API, teal is data, gray is external services or small parts, purple is CI/CD. In the restaurant figures: amber is the builder (fit-out), pink is the running app, teal circles are the numbered concepts.
- **The wide right margin is for your notes.** Figures and tables sit on landscape pages; the evidence log is at the end.

Index of figures

No.	Figure	Belongs to
1	The restaurant: builder, app and the ten key concepts of Program.cs	Sessions 1-2; Days 1-4
2	The restaurant, illustrated	Sessions 2 and 4; Days 1-4, 7
3	Session 3: Entity Framework Core, migrations and seeding	Session 3; Days 6-9
4	Entity, model and DTO compared with Flutter clean architecture	Session 3; Days 2, 7-8
5	Brief Quest system overview	Days 1-20
6	One request's journey through the API	Days 2-4, 13-14
7	Entity class diagram (updated on Day 9)	Days 6-9, 11-12
8	CI/CD pipeline and environments	Days 10, 14, 19

Index of tables and listings

No.	Table	Belongs to
1	HTTP verbs and status codes	Session 2; Days 2, 8-9
2	Words that are easy to confuse	Sessions 2-4; Days 2, 7, 13
3	Common builder services	Sessions 3-4; Days 1-16
4	Flutter clean architecture compared with this C# project	Session 3; Days 2, 7-8
5	Project types: minimal API, controllers and MVC	Day 1
6	Records and DTOs we expect to write	Days 2-15
7	Helper files and classes	Days 2-15
L1	Listing 1: Folder layout	Day 1 onward

Which pictures help on which days

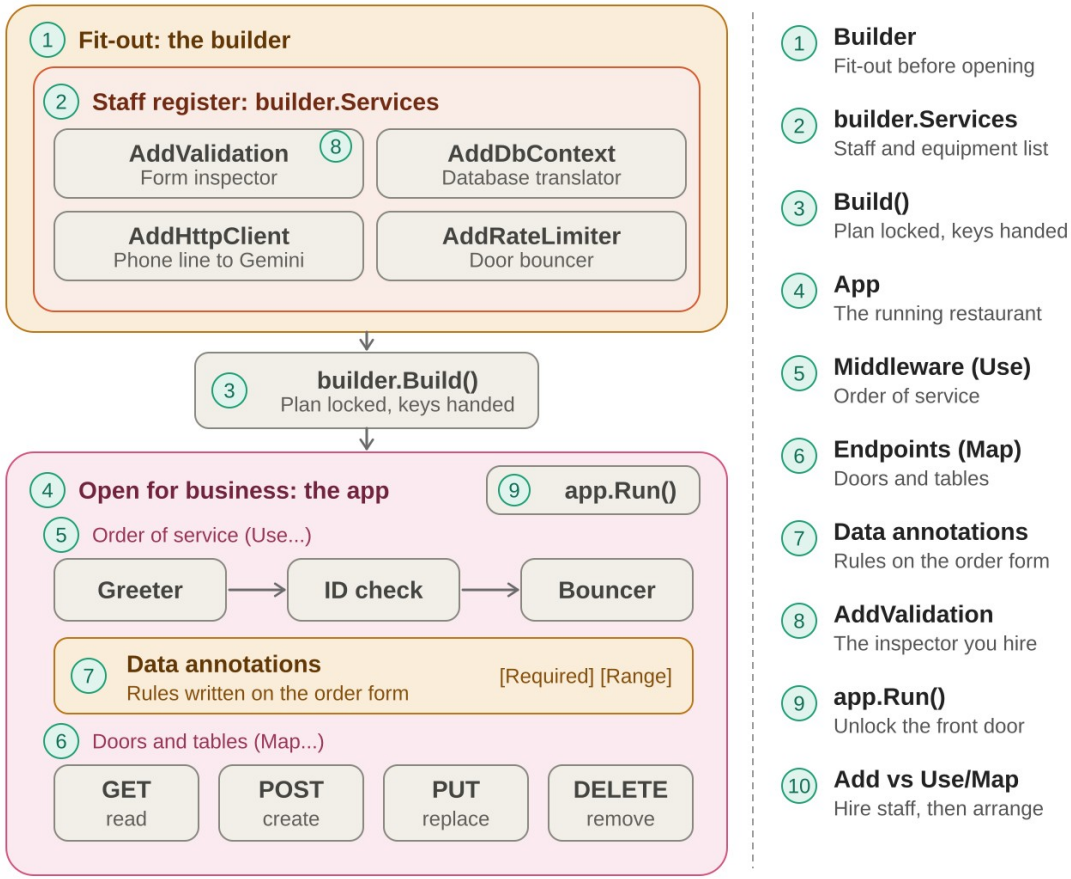
Plan days	Figures	Tables and listings
Days 1-2	1, 2, 5, 6	Tables 1, 2, 5, 6; Listing 1
Days 3-5	1, 6	Tables 3, 6, 7
Days 6-9	3, 4, 7	Tables 2, 3, 4, 6, 7
Day 10	8	Table 7
Days 11-14	5, 6, 7, 8	Tables 3, 6, 7
Days 15-20	5, 8	Tables 6, 7

Part A: Pictures and tables from the tutorial (Sessions 1-4)

The restaurant pictures, the Entity Framework Core session and the comparison with Flutter, followed by tables that sum up the key ideas.

- **Figures 1-4:** the restaurant diagram, the illustrated restaurant, Session 3 (Entity Framework Core), and entity, model and DTO compared with Flutter.
- **Tables 1-5:** HTTP verbs and status codes, words that are easy to confuse, common builder services, Flutter compared with C#, and project types.

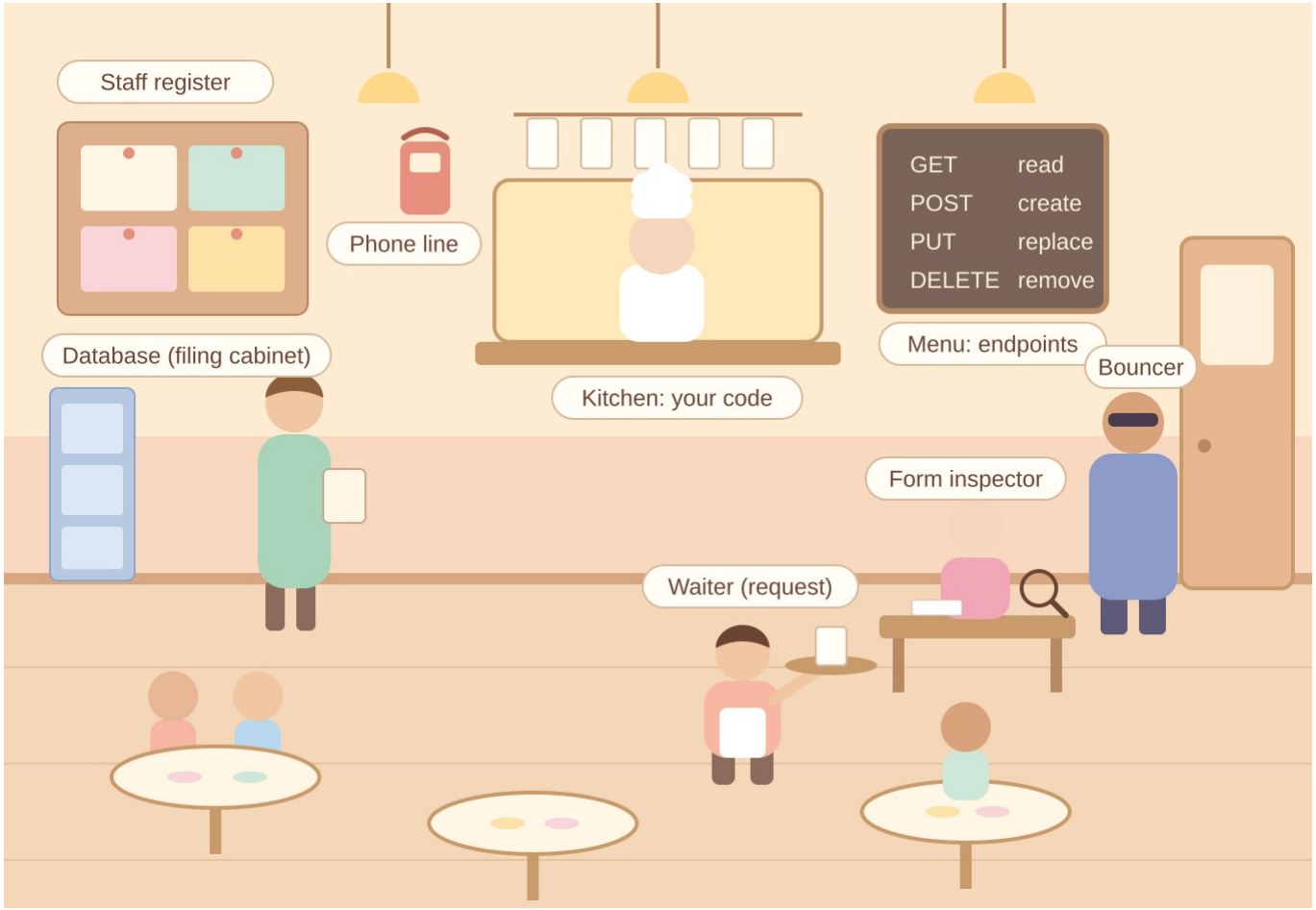
Figure 1. The restaurant: builder, app and the ten key concepts of Program.cs



Description. A colour-coded map of Program.cs as a restaurant in two phases. The builder fits out the restaurant (the staff register, builder.Services), Build() hands over the keys, and the app opens for business (middleware, data annotations, endpoints, app.Run). The numbered list on the right names each concept.

Belongs to. Tutorial Sessions 1-2 (setup, REST API, validation) and the dependency injection chapter; plan Days 1-4.

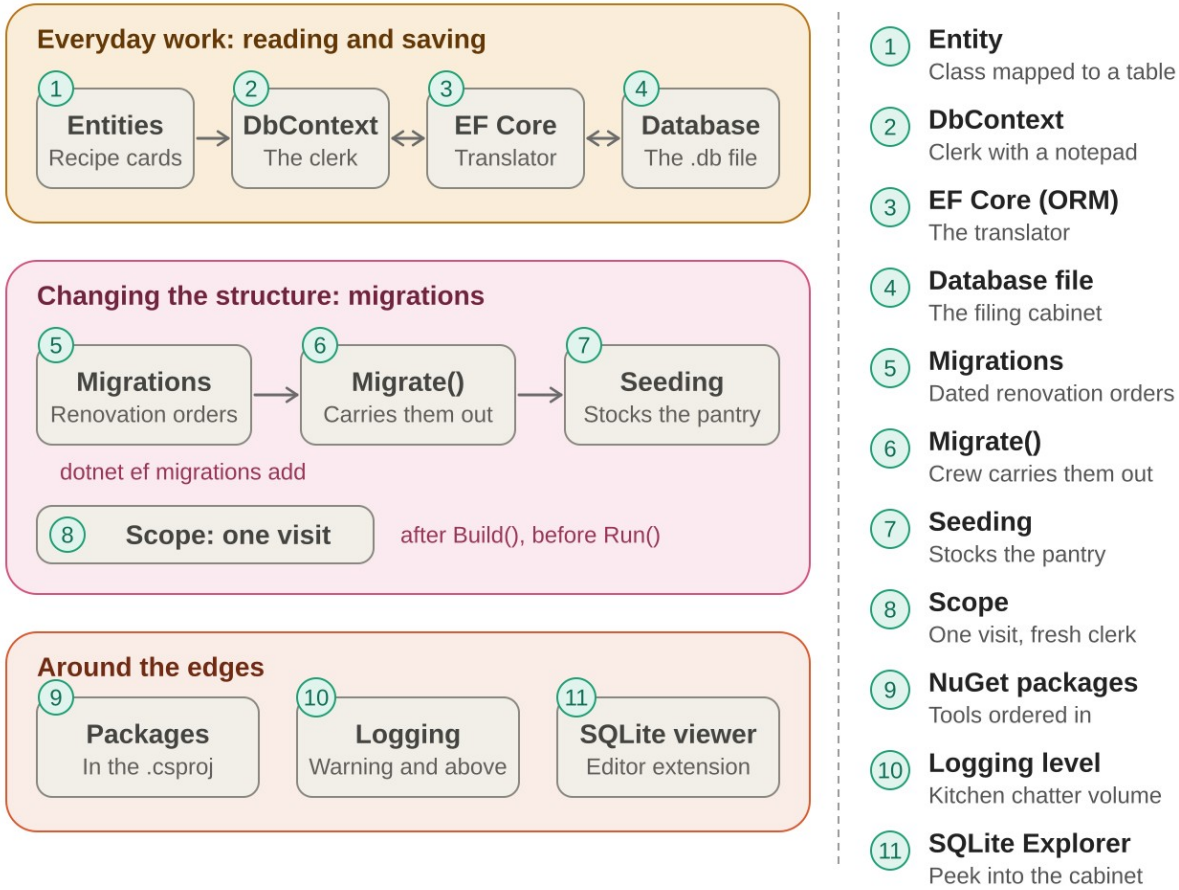
Figure 2. The restaurant, illustrated



Description. An illustrated version of Figure 1: the manager and staff-register board (builder.Services), the inspector (AddValidation), the bouncer (rate limiter), the phone (HttpClient), the filing cabinet (database), the waiter (an HTTP request), the kitchen (your endpoint code) and the chalkboard menu (endpoints: GET, POST, PUT, DELETE).

Belongs to. The same concepts as Figure 1; Sessions 2 and 4; plan Days 1-4 and 7.

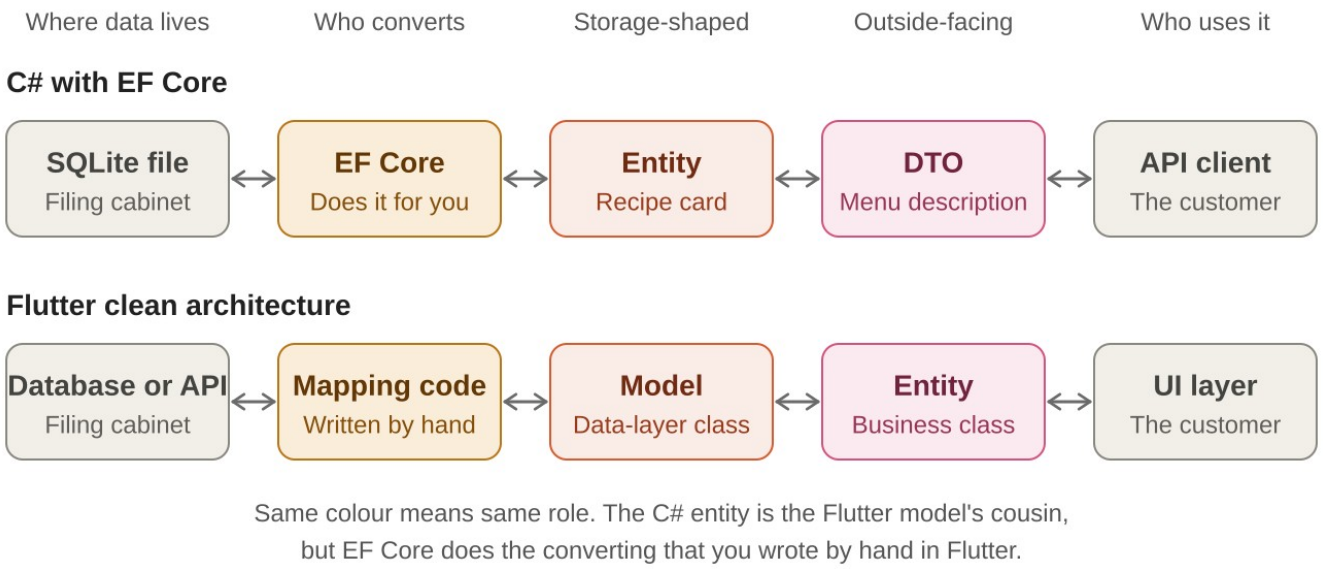
Figure 3. Session 3: Entity Framework Core, migrations and seeding



Description. Three sections: everyday work (entities, DbContext, EF Core, the database file), changing the structure (migrations, Migrate(), seeding, scope), and the tools around the edges (packages, logging level, SQLite viewer). Eleven numbered concepts are listed on the right.

Belongs to. Tutorial Session 3 (1:34:00 to 2:17:45); plan Days 6-9.

Figure 4. Entity, model and DTO compared with Flutter clean architecture



Description. Two lanes show how data travels through a C# project with EF Core and through a Flutter clean-architecture project. Matching colours show matching roles: the C# entity is the cousin of the Flutter model, and EF Core does the converting that Flutter does with hand-written mapping code.

Belongs to. Session 3 follow-up question; plan Days 2 and 7-8 (DTOs and entities).

Table 1. HTTP verbs and status codes

Item	Meaning	Everyday picture
GET	Read data without changing it	Looking at the menu
POST	Create something new (sent twice, you get two)	Handing in a new order form
PUT	Replace the whole resource (repeating changes nothing more)	Reprinting the whole menu card
PATCH	Change part of a resource	Correcting one line with a pen
DELETE	Remove a resource (repeating changes nothing more)	Taking a dish off the menu
200 OK	Success, with a body	The order arrives
201 Created	A new resource now exists, with its location	A new ticket is numbered and pinned up
204 No Content	Success with nothing to return	"Done", nothing more to say
400 Bad Request	The request was invalid	The order form was filled in wrongly
404 Not Found	That resource does not exist	That dish is not on the menu
500 Internal Server Error	Something broke on the server	The kitchen has a problem, not the customer

Belongs to. Session 2; plan Days 2 and 8-9.

Table 2. Words that are easy to confuse

Concept	Everyday picture	Key point
Singleton	One town hall for everyone	One instance for the app's lifetime, created and supplied by dependency injection
Static class	A toolbox you cannot copy	Cannot be instantiated or injected; you call its tools directly
Record	A laminated, filled-in form	Compares by value; make a changed copy with `with`
Class	A house	Compares by identity unless you change that
DTO	Packaging label on a delivery box	Shapes what the API sends and receives
Entity	A recipe card in the kitchen	A class that EF Core maps to a table
DbContext	A clerk with a notepad	Translates, tracks changes and saves on SaveChanges; one per visit
Scoped, Transient, Singleton	One clerk per visit, a new clerk every time, one clerk for the whole restaurant	The lifetimes of services in dependency injection

Belongs to. Sessions 2-4; plan Days 2, 7 and 13.

Table 3. Common builder services

Service	Everyday picture	Where in Brief Quest
AddOpenApi	The printed menu describing what you offer	In the template (Day 1)
AddValidation	The order-form inspector	Input checks (Days 2-4)
AddHttpClient	A phone line to another restaurant	Calling Gemini (Day 3)
AddDbContext / AddSqlite	A translator between kitchen language and the filing cabinet	Database (Day 7)
AddAuthentication / AddAuthorization	Door staff checking IDs and the guest list	Accounts (Days 11-12)
AddRateLimiter	A bouncer limiting customers per minute	Daily cap and limits (Day 13)
AddProblemDetails	A standard format for error slips	Consistent errors (Day 14)
AddCors	The list of other restaurants allowed to send customers	Front end calls (Day 16)

Belongs to. Sessions 3-4; plan Days 1, 3, 7, 11-14 and 16. Exact names are confirmed in the docs when each day arrives.

Table 4. Flutter clean architecture compared with this C# project

Flutter clean architecture	This C# project	Note
Entity (pure business idea)	Usually the same class as the C# entity, unless a separate domain layer is added	Often skipped in small projects
Model (database or API shape, converts itself)	Split in two: the entity (database shape) and the DTO (API shape)	The two roles are separated
Hand-written toMap and fromMap	EF Core does the conversion (the ORM)	Less code to write
Data source or repository	The DbContext, plus a repository class if you add one	Closest match: a database helper or DAO
Mapper between model and entity	Mapping between entity and DTO, often in extension methods	Same idea, different place

Belongs to. Session 3 follow-up; plan Days 2 and 7-8.

Table 5. Project types: minimal API, controllers and MVC

Type	Everyday picture	What it serves	For Brief Quest?
Minimal API (the webapi template)	A food truck with a short menu	Data (JSON) from endpoints	Yes: the plan and the tutorial use it
Controller-based Web API	A restaurant with a manager for each menu section	Data (JSON), organised in controller classes	Not now; a Weeks 5-6 side exercise
MVC web app	A restaurant with its own built-in dining room	Finished HTML pages made by the server	No: Next.js is the dining room

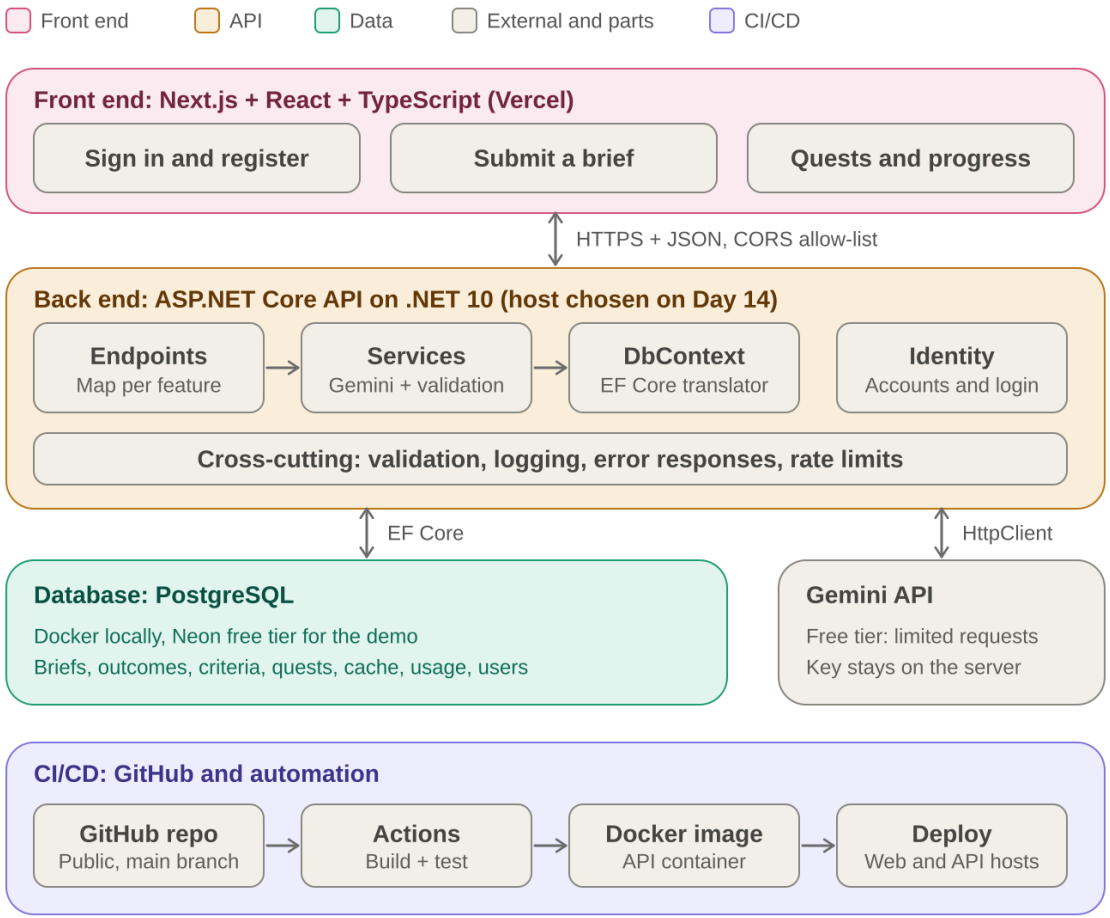
Belongs to. Plan Day 1 (project choice).

Part B: The Brief Quest architecture (plan Days 1-20)

How the pieces fit together, the draft data model, and the helper files and records we expect to write. Anything undecided is labelled with the day it will be decided.

- **Figures 5-8:** the system overview, one request's journey, the draft entity diagram, and CI/CD with environments.
- **Tables 6-7 and Listing 1:** records and DTOs, helper files and classes, and the folder layout.

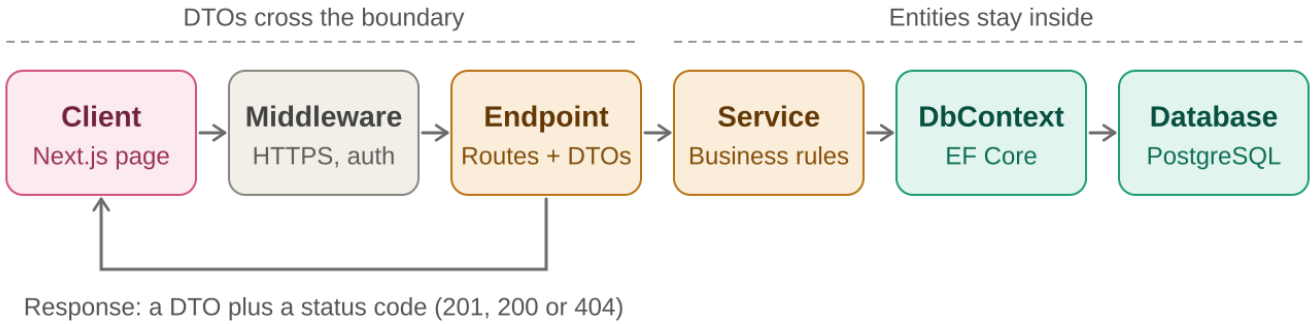
Figure 5. Brief Quest system overview



Description. All the pieces of Brief Quest: the Next.js front end on Vercel, the ASP.NET Core API (endpoints, services, DbContext, Identity), PostgreSQL, the Gemini API, and the CI/CD band (GitHub, Actions, Docker, deploy). Colours: pink front end, amber API, teal data, gray external parts, purple CI/CD.

Belongs to. Whole plan, Days 1-20 (API host decided on Day 14).

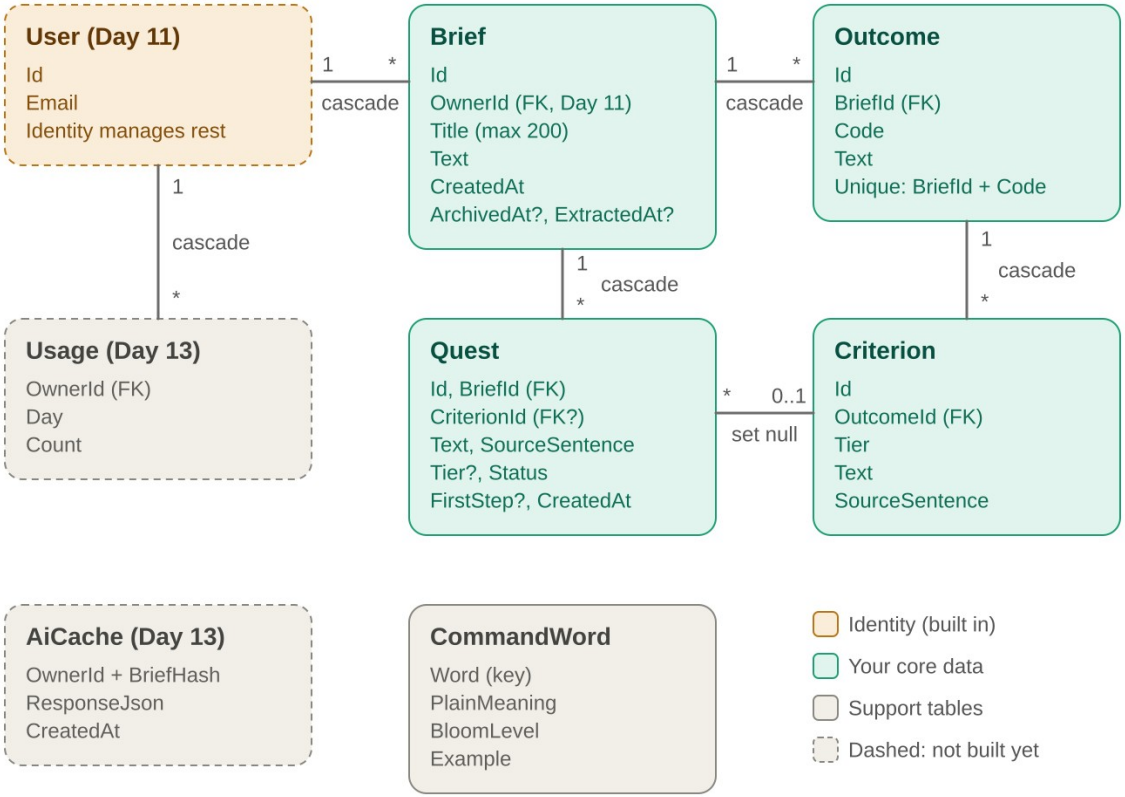
Figure 6. One request's journey through the API



Description. A request travels from the client through middleware, endpoint, service, DbContext and database. DTOs cross the boundary on the left, entities stay inside on the right, and the response returns as a DTO with a status code.

Belongs to. Plan Days 2-4 (endpoints, validation, extraction) and Days 13-14 (daily cap, error responses).

Figure 7. Entity class diagram (updated on Day 9)

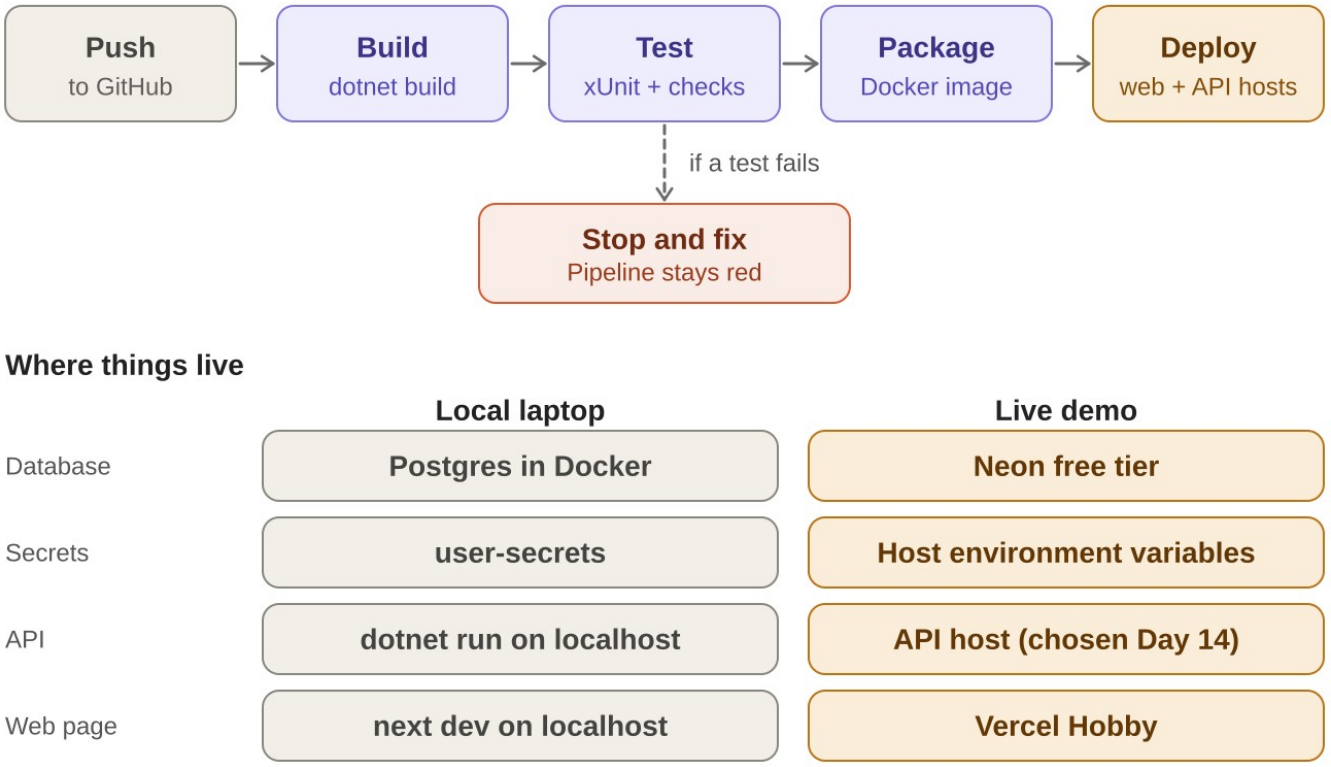


Cascade: deleting the parent deletes its children. Set null: the link is emptied.
OwnerId and the User table arrive with accounts on Day 11.

Description. The tables and how they link (1 means one, * means many), updated after Days 7-9. Solid boxes are built: Brief, Outcome, Criterion, Quest and CommandWord. Dashed boxes are planned: User (Day 11), Usage and AiCache (Day 13). Cascade means deleting the parent deletes its children; set null empties the link. A quest belongs to a brief and may link to one criterion.

Belongs to. Plan Days 6-9 (database, schema agreed on Day 6, updated on Day 9) and Days 11-12 (ownership and accounts).

Figure 8. CI/CD pipeline and environments



Description. Every push triggers build, test, package and deploy; a failed test stops the pipeline. The lower grid compares the local laptop with the live demo for the database, secrets, API and web page.

Belongs to. Plan Day 10 (CI), Day 14 (API host decision) and Day 19 (deployment).

Table 6. Records and DTOs we expect to write

Name	Direction	Fields (draft)	Used by	Day
CreateBriefDto	in	Title, Text	POST /briefs	2 (done)
BriefDto	out	Id, Title, Text	GET /briefs	2 (done)
ExtractedRequirement	internal (AI output)	Text, SourceSentence, Tier	Extraction service	4
BriefDetailDto	out	Brief plus outcomes, criteria and quests	GET /briefs/{id}	8
QuestDto	out	Id, Text, SourceSentence, Status, FirstStep	Quest endpoints	8-9
UpdateQuestStatusDto	in	Status	PATCH quest	9
RegisterDto / LoginDto	in	Email, Password	Account endpoints	11
AuthResponseDto	out	Token details (approach confirmed Day 11)	Login	11
CommandWordDto	out	Word, PlainMeaning, BloomLevel, Example	Decoder	15
ProblemDetails	out	Standard error shape	Every error	14

Belongs to. Plan Days 2-15.

Table 7. Helper files and classes

File or class	What it is	Everyday picture	Day
Program.cs	Entry file	Fit-out, then opening	Done
Endpoints/*Endpoints.cs	One static class per feature, with a MapXEndpoints method	Doors and tables	2 onward
Extensions/ ServiceCollectionExtensions.cs	Methods such as AddBriefQuestDb()	A tidy fit-out checklist	7
Options/GeminiOptions.cs	Settings class filled from configuration (new idea: the options pattern)	A typed settings card	3

File or class	What it is	Everyday picture	Day
Services/GeminiClient.cs	Wraps the HTTP calls to Gemini	The phone operator	3
Services/ExtractionService.cs	Builds the prompt, calls Gemini, parses the result	The chef's routine	3-4
Services/SourceSentenceValidator.cs	Checks every quoted sentence exists in the brief	The fact-checker	4
Services/AiCacheService.cs	Stores and looks up earlier AI answers	The recipe memory	9
Services/UsageCapService.cs	Per-user daily AI limit	The bouncer's tally sheet	13
Data/BriefQuestDbContext.cs	The EF Core context	The clerk	7
Data/Migrations/	Generated migration files	Dated renovation orders	7
Data/SeedData.cs	Command words and starting data	Stocking the pantry	8
Entities/*.cs	Classes mapped to tables	Recipe cards	7-8
api.tests/	xUnit unit and integration tests	The taste testers	5

Belongs to. Plan Days 2-15.

Listing 1. Folder layout

```
brief-quest/
├── api/
│   ├── Endpoints/      (one file per feature)
│   ├── Services/       (Gemini, extraction, validation, cap, cache)
│   ├── Data/           (DbContext, Migrations, SeedData)
│   ├── Entities/       (Brief, Outcome, Criterion, Quest and so on)
│   ├── Dtos/           (records for requests and responses)
│   ├── Options/        (typed settings classes)
│   ├── Extensions/     (tidy-up methods for Program.cs)
│   └── Program.cs
├── api.tests/
├── web/                (Next.js)
├── docker-compose.yml  (local PostgreSQL)
└── .github/workflows/  (CI)
```

Belongs to. Day 1 onward; Entities replaces the Models folder when the database arrives in Week 2.

Evidence log: what I have learnt so far

A summary of the main things learnt and the evidence for each. Add new entries as the plan continues.

When or topic	What I learnt or did	Evidence
Sessions 1-2 (13-15 Oct)	Set up .NET 10 and VS Code; understood dotnet, build versus run and the project structure; learnt REST basics, HTTP verbs (PUT as replace), records for DTOs, first endpoints, .http requests, extension methods and data annotations.	Ran the tutorial project; predicted the tutorial's next step correctly; fixed my F5 key problem (the Fn key).
Builder and app (15 Oct)	Understood WebApplication, the builder (services) and the app (middleware and endpoints), why validation is enabled in the builder, and the Add versus Use/Map rule of thumb.	Sorted AddDbContext, MapGet and AddHttpClient into the right phase, with reasons.
Session 3 (15 Oct)	ORM and Entity Framework Core, DbContext, entities, migrations (writing them versus applying them with Migrate()), seeding, logging levels, the SQLite viewer and scopes.	Asked sharp questions (.csproj folders, migration files, model versus entity) and tested my own answers.
Entity, model, DTO (15 Oct)	Entity versus model versus DTO in C#, and how that differs from Flutter clean architecture.	Compared with my Flutter knowledge (Figure 4, Table 4).
Session 4 (16 Oct)	Dependency injection and lifetimes, IConfiguration and the connection string, async/await, AsNoTracking, Select, Include, GenreDto and endpoint extension methods.	Logged the unfamiliar methods and asked for theory and quizzes.
Self-testing	Completed a whole-tutorial review quiz covering Sessions 1-4.	Scored 37 out of 40 and understood why the three wrong answers were wrong.
Finished the tutorial	Completed the full ASP.NET Core .NET 10 beginner course.	A learning log entry for every session.
Brief Quest Day 1	Created an ASP.NET Core Web API project; read Program.cs, the .csproj, appsettings.json, launchSettings.json and the .http file; added my own endpoint.	Ran it four ways: dotnet run, an .http request, the built-in browser and F5.

When or topic	What I learnt or did	Evidence
Git and GitHub	Created a .NET .gitignore, made a first commit and published a public repository.	Repository online with no bin or obj folders.
Secrets	Stored the AI key with user-secrets, outside the repository; understood configuration layers and environments.	Saw "Key found" in Development and "Key missing" in Production, as I predicted.
Troubleshooting	Worked through the Google AI Studio "permission denied" error.	Fixed it myself using the checklist.
Brief Quest Day 2 (so far)	Wrote DTO records, a grouped endpoint class, in-memory storage and GET and POST endpoints; learnt why IDs should start at 1.	Tested with .http requests; applied review feedback (List instead of ArrayList, CreatedAtRoute, naming).
Architecture	Understood the planned architecture: layers, the DTO boundary, entity relationships, CI/CD and environments.	This pack (Figures 5-8, Tables 6-7).
Ways of learning	Pause and predict, everyday pictures, learning logs, quizzes and one small step at a time.	Logs for Sessions 1-4 and Day 1.